

EXPERIMENT 4

Student Name: Ridhima Gulati

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY701

Section/Group: 24AIT_KRG-G2

Date: 27-07-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, self-joins, and date operations to solve real-world database problems.

Question 1:

(i) You are given a table `tbl_happiness` that contains the ranking of happiness for different countries. Your task is to retrieve all the records from the table while ensuring that "India" and "Sri Lanka" appear at the top, followed by the rest of the countries in their original ranking order.

(ii) Make all the rankings in descending order except India and Sri Lanka.

Table:

```
CREATE TABLE tbl_happiness
```

```
(
```

```
    sno INT PRIMARY KEY,
```

```
    rankings INT,
```

```
    country VARCHAR(50)
```

```
);
```

```
INSERT INTO tbl_happiness
```

```
VALUES
```

```
(1,1,'Finland'),
```

```
(2,2,'Denmark'),
```

EXPERIMENT 4

(3,3,'Iceland'),
 (4,4,'Israel'),
 (5,5,'Netherlands'),
 (6,6,'Sweden'),
 (7,7,'Norway'),
 (8,126,'India'),
 (9,128,'Sri Lanka');

```
SELECT * FROM tbl_happiness;
```

Showing rows: 1 to 9  Page No: 1 of 1			
	sno [PK] integer 	rankings integer 	country character varying (50) 
1	1	1	Finland
2	2	2	Denmark
3	3	3	Iceland
4	4	4	Israel
5	5	5	Netherlands
6	6	6	Sweden
7	7	7	Norway
8	8	126	India
9	9	128	Sri Lanka

Answer: (i)
 SELECT rankings,
 country
 FROM tbl_happiness
 ORDER BY
 CASE
 WHEN country = 'India' THEN 1
 WHEN country = 'Sri Lanka' THEN 2
 ELSE 3

EXPERIMENT 4

END,
rankings;

	rankings integer	country character varying (50)
1	126	India
2	128	Sri Lanka
3	1	Finland
4	2	Denmark
5	3	Iceland
6	4	Israel
7	5	Netherlands
8	6	Sweden
9	7	Norway

```
(ii) SELECT rankings,
       country
FROM tbl_happiness
ORDER BY
CASE
    WHEN country = 'India' THEN 1
    WHEN country = 'Sri Lanka' THEN 2
    ELSE 3
END,
CASE
    WHEN country IN ('India', 'Sri Lanka') THEN rankings
END ASC,
CASE
```

EXPERIMENT 4

WHEN country NOT IN ('India', 'Sri Lanka') THEN rankings
END DESC;

	rankings integer 	country character varying (50) 
1	126	India
2	128	Sri Lanka
3	7	Norway
4	6	Sweden
5	5	Netherlands
6	4	Israel
7	3	Iceland
8	2	Denmark
9	1	Finland

Question 2: A company maintains employee information in a single table where each employee may report to another employee acting as their manager. Prepare a report that displays the details of every employee who supervises at least one other employee. The report should include the manager's employee ID, name, total number of direct reports, and the average age of those reporting employees. The average age should be rounded to the nearest whole number and the final result should be sorted by employee ID.

Table: (i)

```
CREATE TABLE Employees
(
  employee_id INT PRIMARY KEY,
  name VARCHAR(50),
  reports_to INT,
  age INT
```

EXPERIMENT 4

);

```
INSERT INTO Employees (employee_id, name, reports_to, age)
VALUES
```

```
(9, 'Hercy', NULL, 43),
```

```
(6, 'Alice', 9, 41),
```

```
(4, 'Bob', 9, 36),
```

```
(2, 'Winston', NULL, 37);
```

```
SELECT * FROM Employees;
```

	employee_id [PK] integer	name character varying (50)	reports_to integer	age integer
1	9	Hercy	[null]	43
2	6	Alice	9	41
3	4	Bob	9	36
4	2	Winston	[null]	37

Answer:

```
SELECT
```

```
    m.employee_id,
```

```
    m.name,
```

```
    COUNT(e.employee_id) AS reports_count,
```

```
    ROUND(AVG(e.age), 0) AS average_age
```

```
FROM Employees AS m
```

```
JOIN Employees AS e
```

```
    ON m.employee_id = e.reports_to
```

```
GROUP BY
```

```
    m.employee_id,
```

```
    m.name
```

```
ORDER BY
```

```
    m.employee_id;
```

EXPERIMENT 4

	employee_id [PK] integer	name character varying (50)	reports_count bigint	average_age numeric
1	9	Hercy	2	39

LEARNING OUTCOMES:

1. Learned to use conditional logic with the CASE statement to customize the display and sorting of query results according to specific business requirements.
2. Gained hands-on experience in sorting data using multiple conditions with the ORDER BY clause, including implementing both ascending and descending order for selected records.
3. Learned to apply aggregate functions such as COUNT() and AVG() along with ROUND() to generate meaningful summary reports from employee data.
4. Developed an understanding of self-joins by joining a table with itself to represent hierarchical relationships, such as managers and their direct reporting employees.
5. Enhanced the ability to design optimized SQL queries that combine conditional expressions, aggregate functions, grouping, and self-joins to solve real-world business problems and generate accurate reports.